

학습 결과서 — 머신러닝 파트

담당 B · 데이터 전처리 v1.2 · 작성 2026-08-28

딥러닝 파트는 docs/02_학습결과서.md 를 보세요. 이 문서는 머신러닝 5종의 튜닝과 최종 모델 선정을 다룹니다.

한 장 요약

머신러닝 5종을 하이퍼파라미터 튜닝하고, **학습에 한 번도 쓰지 않은 게임 12개**로 최종 검증했습니다.

질문	답
성능이 가장 좋은 모델은?	랜덤포레스트. 봉인 12게임에서 AUC 0.7232
CV 1위였던 XGBoost는?	봉인에서 0.7029 로 3위까지 떨어짐
튜닝이 얼마나 올렸나?	게임분할 기준 +0.003 ~ +0.008 . 작다
불균형 처리(SMOTE-balanced)가 필요한가?	아니다 . 세 방식 best F1 차이가 0.0030
그럼 뭐가 효과가 컸나?	임계값 . 0.5 → 최적점으로 F1 +0.0560 (19배)
모델이 게임 이름을 외웠나?	외웠다. 랜덤 0.8104 → 봉인 0.7232, 격차 0.0872
그럼 왜 그걸 배포 안 했나?	성능 0.009를 내주고 학습 11배 속도와 SHAP 1:1 대응 을 샀다 (3절)
배포 모델은?	LightGBM · 봉인 12게임 AUC 0.7187 · 임계값 0.2899
배포 직전에 발견한 문제는?	확률이 부풀려져 있었다 . class_weight 를 빼서 고쳤다 (7절)

아래 숫자는 전부 results/tuned_results.csv 등 실제 실행 기록에서 그대로 옮긴 값입니다.

읽기 전에 — 2·5·6절의 표는 **튜닝 실험 기록**입니다. 실제로 배포한 모델은 그 결과에서 class_weight 하나를 뺀 것이라 숫자가 다릅니다. 배포본은 3절에 따로 정리했습니다. 왜 뺐는지는 7절 「확률이 부풀려져 있었다」를 보세요.

1. 실험 설계

왜 게임 단위로 잘랐나

게임마다 이탈률이 **9.7% ~ 88.8%** 로 9배 차이 납니다. 데이터를 그냥 섞어 나누면

모델이 **게임 이름만 외워도** 점수가 오릅니다. 그래서 게임 단위로 자르고,

튜닝에 쓸 조각과 마지막에 딱 한 번 열 조각을 미리 갈라뒀습니다.

전체 게임 60개
└─ 튜닝용 48개 · 111,759행 · 이탈률 40.00%
| └─ 안쪽 GroupKFold(4) ─ 검증도 '처음 보는 게임'
└─ 봉인 12개 · 27,899행 · 이탈률 45.58% ← 마지막 한 번

겹치는 게임 **0개**. 봉인 조각은 GroupKFold(5) 의 1조각으로 코드에 고정했습니다 —

"이 조각은 별로니 다른 조각으로" 를 하는 순간 봉인이 깨지기 때문입니다.

지킨 규칙 네 가지

	규칙	어떻게
1	학습 전 누수 검사	열 목록을 dataset_meta.json 에서 읽고 assert_no_leak() 로 3중 검사. 손으로 타이핑한 열 이름이 한 곳도 없음
2	라벨 재생성 금지	dataset.csv 의 churn 을 그대로 읽음. 튜닝 코드에 라벨 계산이 한 줄도 없음
3	두 분할 모두 보고	랜덤 80/20 · 게임 5조각 · 봉인, 세 줄로 기록
4	선택 기준 사전 고정	scoring="roc_auc", refit=True — sklearn 이 안쪽 CV 평균 AUC 최고를 자동 선택. 결과를 보고 고를 여지 없음

탐색 방법

격자 탐색을 고집하지 않았습니다. 연속값이 섞인 공간에서는 같은 예산이면

무작위 탐색이 좋은 값을 더 잘 찾습니다.

모델	방법	후보	학습 횟수	탐색 시간	고른 이유
로지스틱	GridSearchCV	6	24	52.7s	후보가 6개뿐 — 전수 탐색이 더 싼
랜덤포레스트	RandomizedSearchCV	12	48	991.1s	1회 학습이 비쌘
XGBoost	RandomizedSearchCV	30	120	561.5s	연속값 7개가 섞인 넓은 공간
LightGBM	RandomizedSearchCV	30	120	380.8s	위와 같음
MLP	RandomizedSearchCV	10	40	367.3s	1회 약 14초

총 **352회 학습 · 39분**. 후보 전원의 조각별 점수는 results/tuning/ 에 남겼습니다.

2. 최종 결과

분할방식마다 그 숫자를 얼마나 믿을 수 있는지가 다릅니다.

배포 성능의 오염 없는 추정치는 **봉인** 한 줄뿐입니다.

- **봉인(게임12)** — 오염 없음. 튜닝·임계값 어디에도 쓰지 않음
- **게임(5)** — 부분 오염. 봉인 게임까지 포함한 전체 데이터로 계산
- **랜덤** — 다른 질문. 낮아서가 아니라 재는 대상이 '아는 게임에서의 실력'

모델명	변수묶음	분할방식	AUC	PR-AUC	Recall	F1	학습시간	전처리버전
로지스틱(튜닝)	B셋	봉인(게임 12)	0.6363	0.5914	0.9421	0.6336	1.8s	v1.2
로지스틱(튜닝)	B셋	랜덤	0.7645	0.6755	0.8686	0.6611	1.8s	v1.2
로지스틱(튜닝)	B셋	게임(5)	0.7147	0.6342	0.8335	0.6372	8.1s	v1.2
랜덤포레스트(튜닝)	B셋	봉인(게임 12)	0.7232	0.6674	0.9583	0.6508	20.7s	v1.2
랜덤포레스트(튜닝)	B셋	랜덤	0.8104	0.7479	0.8456	0.6926	24.5s	v1.2
랜덤포레스트(튜닝)	B셋	게임(5)	0.7575	0.6848	0.8288	0.6626	105.5s	v1.2
XGBoost(튜닝)	B셋	봉인(게임 12)	0.7029	0.6592	0.9196	0.6502	2.4s	v1.2
XGBoost(튜닝)	B셋	랜덤	0.7998	0.7325	0.8066	0.6884	2.5s	v1.2
XGBoost(튜닝)	B셋	게임(5)	0.7493	0.6767	0.7910	0.6611	12.3s	v1.2
LightGBM(튜닝)	B셋	봉인(게임 12)	0.7209	0.6769	0.9218	0.6503	1.8s	v1.2
LightGBM(튜닝)	B셋	랜덤	0.8180	0.7566	0.8517	0.6997	1.8s	v1.2
LightGBM(튜닝)	B셋	게임(5)	0.7479	0.6772	0.8074	0.6524	9.5s	v1.2
MLP(튜닝)	B셋	봉인(게임 12)	0.6717	0.6252	0.9308	0.6411	11.8s	v1.2
MLP(튜닝)	B셋	랜덤	0.8104	0.7422	0.8677	0.6890	9.7s	v1.2
MLP(튜닝)	B셋	게임(5)	0.7185	0.6419	0.8154	0.6316	50.1s	v1.2

임계값 안내 — 다른 줄과 비교할 때 주의 위 15줄의 F1-Recall 은 모델마다 다른 임계값에서 낸 값입니다 (로지스틱 0.3523 · 랜덤포레스트 0.4015 · XGBoost 0.4338 · LightGBM 0.3735 · MLP 0.2521). 배포되는 모델이 실제로 그 값으로 판정하므로, 표에도 같은 조건을 적었습니다. 임계값은 results.csv 의 **메모 열**에 줄마다 적혀 있습니다. 반면 results.csv 의 **다른 줄**(기본값 32줄 · 딥러닝 18줄)은 evaluate.py 규칙대로 **임계값 0.5**로 낸 값입니다. **F1-Recall**을 가로질러 비교하지 마세요. **AUC-PR-AUC**는 임계값과 무관하므로 66줄 전체에서 그대로 비교됩니다.

설정을 고른 기준 — 안쪽 CV

모델	안쪽 CV AUC	편차	후보 최저	후보 중앙	후보 최고	임계값(OOF)
XGBoost	0.7687	±0.0435	0.7095	0.7503	0.7687	0.4338
랜덤포레스트	0.7613	±0.0474	0.7558	0.7583	0.7613	0.4015
LightGBM	0.7540	±0.0424	0.7101	0.7449	0.7540	0.3735
MLP	0.7408	±0.0447	0.7002	0.7195	0.7408	0.2521
로지스틱	0.7299	±0.0449	0.7148	0.7174	0.7299	0.3523

편차가 ±0.042 ~ 0.047 입니다. 상위 3종(0.7687 / 0.7613 / 0.7540)의 차이는 이 편차 안에 들어갑니다 — CV 만으로는 우열을 못 가립니다. 그래서 봉인 결과가 필요했습니다.

3. 최종 모델과 배포 모델

튜닝이 고른 것 — 랜덤포레스트

게임 기준 세 측정 모두에서 1~2위인 유일한 모델입니다.

측정	랜덤포레스트	순위
봉인 12게임	0.7232	1위
게임(5) 전체	0.7575	1위
안쪽 CV	0.7613	2위

CV 1위였던 XGBoost(0.7687)는 봉인에서 0.7029로 3위까지 내려갔습니다.

```
{ "class_weight": "balanced_subsample", "max_depth": 12, "max_features": "sqrt",  
  "min_samples_leaf": 3, "n_estimators": 543 }
```

배포한 것 — LightGBM (class_weight 제거)

두 가지를 바꿨습니다. 모델을 LightGBM으로 골랐고, 튜닝이 고른

class_weight="balanced" 를 뺐습니다.

① 왜 LightGBM인가 — 성능 0.009를 내주고 두 가지를 샀습니다

양쪽 다 class_weight 를 뺐 공정한 조건에서, 48게임 4조각으로 비교했습니다

(봉인 미사용).

	LightGBM	랜덤포레스트
AUC	0.7541	0.7630 (+0.0089)
확률 오차 (절대평균)	4.1%	2.3%
학습 시간	1.8초	20.2초 (11배)
SHAP 설명	24개 변수 1:1	원핫이라 쪼개짐

비슷한 숫자가 여럿이라 헷갈리기 쉽습니다. LightGBM 의 48게임 성적이 세 군데 나오는데 전부 다른 계산입니다. | 값 | 무엇 | 어디 | |---|---|---| | 0.7540 | 튜닝이 고른 설정(balanced 포함)의 안쪽 CV | 2절 표 | | **0.7541** | balanced 를 뺐 뒤 4조각 AUC 평균 | 위 표 | | 0.7536 | 같은 모델의 out-of-fold 를 한 덩어리로 채점 | ml_meta.json | 앞의 둘은 조각별 AUC 를 평균한 것이고 마지막은 예측을 모두 모아 한 번에 채점한 것입니다. 우연히 값이 가까울 뿐 같은 숫자가 아닙니다.

랜덤포레스트가 4조각 중 3조각에서 이겼고 차이는 **1.48σ** 입니다.

2σ 문턱 아래라 "확실하다" 고는 못 하지만, "차이가 없다" 고 할 수도 없습니다.

그래서 근거를 이렇게 씁니다 — **AUC 0.009와 확률 정확도 1.8%p를 내주고, 학습 11배 속도와 SHAP이 24개 변수에 1:1로 대응하는 설명 가능성을 얻었습니다.**

화면 1이 "왜 그렇게 판단했나" 를 보여줘야 하는데, 랜덤포레스트는 원핫을 거쳐

SHAP 값이 genre_group=협동 처럼 쪼개져 나와 다시 합치는 코드가 필요합니다.

② 왜 class_weight를 뺐는가 — 확률이 부풀려져 있었습니다

자세한 내용은 7절에 있습니다. 요약하면, 튜닝 기준이 AUC 하나였는데 AUC는

순위만 보므로 확률이 통째로 위로 밀려도 안 떨어집니다. 그래서 확률을 부풀리는 설정이 뺐혔고, 화면은 확률 숫자를 그대로 보여주므로 그대로 노출됐습니다.

	예측 평균	실제	왜곡
balanced 있음	60.4%	45.6%	+ 14.9%p
balanced 제거	52.4%	45.6%	+6.8%p

models/ml_best_params.json 은 고치지 않았습니다 — 그건 "튜닝이 무엇을 골랐나" 의 기록입니다. 배포에서 무엇을 왜 바꿨는지는 models/ml_meta.json 의 배포에서_변경 에 남겼습니다.

배포본 성적표 — 봉인 12게임 (5회차 개봉)

지표	값	비교 대상
AUC	0.7187	무작위 0.5
PR-AUC	0.6749	무작위 0.4558
F1	0.6510	전부-이탈 기준선 0.6262 (+0.0248)
Recall	0.9160	—
확률 왜곡	+6.8%p	예측 52.4% vs 실제 45.6%
이탈 판정 비율	82.7%	실제 45.6%

이전 모델(balanced) 대비 AUC는 0.7209 → 0.7187로 **0.0022 내려갔지만**, 확률 왜곡이 **14.9% → 6.8%로 절반 이하**가 됐습니다. 화면이 확률을 그대로 보여주므로 맞는 교환이고, 0.0022는 조각 간 편차(± 0.04)에 묻힙니다.

배포본 설정 (models/ml_meta.json)

```
{
  "class_weight": null,
  "colsample_bytree": 0.8371,
  "learning_rate": 0.02490,
  "min_child_samples": 193,
  "n_estimators": 322,
  "num_leaves": 31,
  "reg_lambda": 7.3387,
  "subsample": 0.7471,
  "subsample_freq": 1
}
```

임계값 **0.2899** — 이 설정으로 튜닝용 48게임의 CV out-of-fold 에서 다시 골랐습니다 (봉인 미사용). **설정이 바뀌면 확률 분포가 달라지므로 옛 임계값 0.3735를 그대로 쓰면 안 됩니다.** 모델과 임계값은 한 세트입니다.

4. 랜덤 분할과 게임 분할의 차이

두 숫자의 차이가 "모델이 게임을 얼마나 외웠나"입니다.

모델	랜덤	게임(5)	봉인(게임12)	랜덤 게임5	랜덤 봉인
랜덤포레스트	0.8104	0.7575	0.7232	0.0529	0.0872
XGBoost	0.7998	0.7493	0.7029	0.0505	0.0969
LightGBM	0.8180	0.7479	0.7209	0.0701	0.0971
로지스틱	0.7645	0.7147	0.6363	0.0498	0.1282
MLP	0.8104	0.7185	0.6717	0.0919	0.1387

- **MLP가 가장 많이 외웠습니다.** 랜덤에서 랜덤포레스트와 똑같은 0.8104인데 봉인에서 0.6717로 떨어집니다
- **랜덤포레스트가 가장 덜 외웠습니다** (격차 0.0872)
- 랜덤 순위로는 LightGBM이 1위지만, **처음 보는 게임을 만나면 순위가 뒤집힙니다**

발표용 문장 "아는 게임에서는 AUC 0.81이 나옵니다. 학습에 한 번도 안 쓴 게임 12개를 넣으면 0.72로 떨어집니다. 이 0.09가 모델이 게임을 외운 몫이고, 저희는 그 숫자를 숨기지 않고 같이 보고합니다."

5. 튜닝이 실제로 얼마나 올렸나

모델	게임(5) 기본값	게임(5) 튜닝	변화	랜덤 기본값	랜덤 튜닝	변화
XGBoost	0.7409	0.7493	+0.0084	0.8171	0.7998	0.0173
LightGBM	0.7405	0.7479	+0.0074	0.8179	0.8180	+0.0001
로지스틱	0.7101	0.7147	+0.0046	0.7649	0.7645	0.0004
랜덤포레스트	0.7545	0.7575	+0.0030	0.8171	0.8104	0.0067

튜닝 이득은 게임분할에서 **+0.003 ~ +0.008**로 작습니다. 대신 랜덤 분할에서는 오히려 떨어졌습니다.

게임 CV를 기준으로 튜닝했으니 당연한 결과입니다 — 처음 보는 게임에 맞추면 아는 게임 성능은 조금 내줍니다.

의도한 교환이고, 배포 조건이 '처음 보는 게임'이므로 맞는 방향입니다.

표를 볼 때 주의 — 튜닝 행의 F1·Recall이 기본값보다 훨씬 높은 건 모델이 좋아져서가 아니라 **임계값이 바뀌어서**입니다. 예: LightGBM 게임(5) — 기본값은 임계값 0.5에서 Recall 0.5871 / F1 0.6027, 튜닝본은 임계값 0.3735에서 Recall 0.8074 / F1 0.6524. **AUC만 임계값과 무관하게 비교 가능합니다.**

6. 불균형 처리와 임계값

LightGBM 고정 · 튜닝용 111,759행 · 안쪽 4조각. 세 방식 모두 같은 원핫 전처리를 거칩니다.

방식	AUC	PR-AUC	F1 @ 0.5	best F1	최적 임계값	시간
기본	0.7515	0.6628	0.5897	0.6457	0.2942	7.0s
class_weight=balanced	0.7477	0.6600	0.6212	0.6440	0.3569	7.3s

방식	AUC	PR-AUC	F1 @ 0.5	best F1	최적 임계값	시간
SMOTE	0.7527	0.6629	0.6105	0.6470	0.2977	25.5s

결론: 불균형 처리는 필요 없습니다. 임계값이 훨씬 중요합니다

- 세 방식의 best F1이 **0.6440 ~ 0.6470** 으로 사실상 같습니다 (차이 **0.0030**)
- 반면 임계값을 0.5에서 최적점으로 옮기면 F1이 **0.5897 → 0.6457** 로 **+0.0560** 오릅니다
- 임계값 조정 효과가 불균형 처리 차이의 **19배**
- SMOTE는 3.6배 느린데(25.5s vs 7.0s) 얻는 게 없습니다
- 우리 데이터는 이탈률 41.1% 로 **1:1.43** 이라 애초에 심한 불균형이 아니었습니다

SMOTE는 imblearn.Pipeline 안에 넣어 각 조각의 학습 쪽에만 적용했습니다.

나누기 전에 걸면 합성된 이웃이 검증 쪽에 새어 점수가 부풀려집니다.

한계 — 원핫 열을 보간하면 0과 1 사이 값이 생깁니다. SMOTE의 알려진 한계이고, 이 비교는 "트리 모델에 SMOTE가 필요한가"를 보는 용도입니다.

7. 솔직하게 — 한계

확률이 부풀려져 있었다 (고쳤습니다)

화면 1이 대체로 80%대만 뵈는다는 제보로 발견했습니다.

튜닝의 선택 기준이 AUC 하나였던 것이 원인입니다. AUC는 순위만 봅니다.

확률이 통째로 위로 밀려도 순위는 그대로라 AUC가 안 떨어집니다. 그래서

확률을 부풀리는 `class_weight="balanced"` 가 땀혔고, **5종 중 4종이 같은 상태였습니다** (MLP만 이 옵션이 없어 빠졌습니다).

모델	봉인에서 예측 평균	실제 45.6% 대비
로지스틱	63.2%	+17.6%p
XGBoost	62.5%	+17.0%p
LightGBM	60.4%	+14.9%p
랜덤포레스트	58.7%	+13.1%p
MLP	55.1%	+9.5%p

배포본에서는 `class_weight` 를 빼서 **+6.8%p**로 줄였습니다. 다만 완전히

없어지는 못했습니다 — 아래 「확률 오차가 조각마다 흔들린다」를 보세요.

교훈은 이겁니다. 화면이 확률 숫자를 보여준다면 선택 기준에 AUC만 두면 안 됩니다. 확률이 실제와 맞는지(보정)를 같이 봐야 합니다.

F1은 "전부 이탈" 기준선과 같이 봐야 한다

봉인 12게임은 이탈률이 45.6%입니다. 여기서 아무것도 안 보고 "전부 이탈" 이라고만 찍어도 F1이 나옵니다.

정밀도 = 45.6% (전부 이탈이라 했으니 실제 이탈 비율 그대로)
 재현율 = 100% (하나도 안 놓침)
 $F1 = 2 \times 0.456 \times 1.0 \div (0.456 + 1.0) = 0.626$

	F1	기준선 대비
전부 이탈이라고 찍기	0.6262	—
배포본 LightGBM	0.6510	+0.0248

F1만 보면 우리 모델이 별로 나은 게 없어 보입니다. 이탈률이 40~46%로 높으면 F1이라는 지표 자체가 그렇습니다. 팀 규칙이 "정확도를 안 쓴다"고 정한 것과 같은 함정이고, 방향만 반대입니다 (정확도는 "전부 잔존"으로 뚫립니다).

AUC는 이 함정에 안 걸립니다. 전부 이탈이라고 찍으면 AUC는 정확히 0.5입니다.

순위를 전혀 안 매기니까요. **주지표를 AUC로 둔 것이 맞았습니다.**

그래서 F1을 보고할 때는 항상 기준선을 같이 적습니다.

확률 오차가 조각마다 흔들린다

class_weight 를 빼도 오차가 0이 되지는 않습니다.

게임 조각	예측 평균	실제	오차
1 (봉인)	52.4%	45.6%	+6.8%
2	37.0%	37.3%	0.2%
3	51.3%	45.3%	+6.0%
4	33.6%	36.9%	3.3%
5	37.0%	40.6%	3.6%

이탈률이 높은 게임 묶음에서 6~7%p 과다 예측하는 경향이 있습니다.

방향은 맞히는데 폭을 과장합니다.

CalibratedClassifierCV 같은 확률 보정으로는 못 고칩니다. 보정은 **일관된 편향**을 고치는 도구인데, 평균 편향은 이미 +1.1%로 작고 남은 것은

조각별 흔들림이기 때문입니다. 게임이 60개뿐인 데서 오는 한계입니다.

화면에 표시할 때는 이렇게 밝히는 것이 정직합니다 —

"처음 보는 게임에서는 확률이 6~7%p 높게 나올 수 있습니다."

봉인 한 조각의 숫자는 크게 흔들린다

봉인은 12게임 한 조각입니다. 5조각을 다 재보면 AUC가 이렇습니다.

조각	LightGBM	랜덤포레스트
1 (우리가 쓴 것)	0.7209	0.7232
2	0.7603	0.7575
3	0.6925	0.6977
4	0.8143	0.8273

조각	LightGBM	랜덤포레스트
5	0.7517	0.7818

0.69에서 0.83까지 벌어집니다. 게임이 60개뿐이고 게임별 이탈률이

9.7~88.8%로 극단적이라, 어떤 12개가 시험지가 되느냐가 크게 좌우합니다.

우리가 쓴 1조각은 중간쯤이었습니다.

봉인 숫자 하나만 보지 말고 게임(5) 5조각 평균(0.7479)을 같이 보셔야 합니다.

봉인을 다섯 번 열었습니다

회차	무엇에	성격
1	5모델 최종 비교	계획된 사용
2	results.csv 합치며 재측정	같은 값
3	결함 감사 — 정밀도·확률왜곡 확인	계획 외
4	조각별 왜곡 진단	계획 외
5	배포본 최종 성적표	계획된 사용

설정 탐색에는 한 번도 쓰지 않았지만, 3·4회차는 진단 목적으로 계획 없이

연 것입니다. 그만큼 봉인의 "한 번도 안 본 데이터" 성격이 약해졌습니다.

추가 실험을 한다면 새 조각이 필요합니다.

그 밖에

게임(5) 행은 봉인 게임까지 포함한 전체 데이터로 계산됩니다. 규칙 3을 지키려고 넣었지만,

하이퍼파라미터는 그중 48개 게임으로 골랐으므로 완전히 독립적인 숫자는 아닙니다.

튜닝은 B셋만 했습니다. A셋(게임 이름 포함)은 게임분할에서 확률이 무너지므로

(실제 이탈률 37%인 조각에 평균 70%를 뺌) 배포 후보가 아니라 제외했습니다.

MLP는 class_weight 를 지원하지 않아 불균형 보정 없이 탐색했습니다.

나머지 4종은 탐색 공간에 포함했습니다 (XGBoost는 scale_pos_weight).

8. 데이터 누수는 없었나

preprocess.py 의 LEAK_COLS 10개가 dataset.csv 에 아예 들어있지 않고,

학습 직전에 assert_no_leak() 이 3중으로 검사합니다.

```
cols = dict(변수목록들())["B셋"] # dataset_meta.json 에서 읽음
assert_no_leak(cols) # 1차
assert "churn" not in cols
X = features(d, cols) # features() 가 FORBIDDEN 으로 2차
assert_no_leak(list(X.columns)) # 3차
```

누수가 있으면 AUC 0.99가 나옵니다. 최고 성능이 **0.8180**(랜덤), **0.7232**(봉인)에

머문 것 자체가 누수가 없다는 방증입니다.

9. 산출물

코드

파일	내용
src/tune_ml.py	하이퍼파라미터 튜닝 + 봉인 검증 + 불균형 실험
src/explain_ml.py	머신러닝 SHAP + 배포 모델 저장
src/train_ml.py	ML 4종 × 4칸 기본값 채점표
src/demo.py	모델링 과정을 한 단계씩 보여주는 학습용 스크립트

실험 기록

파일	내용
results/results.csv	팀 공용 표 66줄 (기본값 32, 딥러닝 18, 튜닝 15, 배포본 1)
results/tuned_results.csv	위 최종표 15줄 (튜닝 임계값 기준)
results/imbalance.csv	불균형 3방식 비교
results/tuning/*.csv	후보 88개 전원의 조각별 점수 (검증용)

모델 부속

파일	내용
models/ml_best_params.json	5종 최고 설정 + CV점수 + 임계값 + 봉인조각 번호
models/ml_feature_order.json	열 순서 · 범주값 · 자료형
models/ml_threshold.json	0.2899 — 배포본 설정으로 48게임 OOF 에서 고른 값
models/ml_meta.json	배포본 설정 · 성능 · 배포에서_변경 · 한계
models/ml_model.joblib	배포본 · LightGBM(class_weight 제거) · <code>git add -f</code>

그림

파일	내용
reports/figures/06_ML_변수중요도.png	SHAP 상위 12개 변수
reports/figures/07_ML_SHAP분포.png	무엇이 이탈 쪽으로 미는가

10. 팀에 전달할 것

담당	내용
전처리	featurize() 가 release_year 를 문자열로 주는데 학습 때는 CSV 를 읽으며 pandas 가 int64 로 파싱합니다. 화면에서 LightGBM 이 죽는 원인이었습니다. 현재는 ml_feature_order.json 에 자료형을 저장해 우회 중입니다
딥러닝	처음엔 두 사람이 explain.py 라는 같은 이름으로 만들어 겹쳤습니다. 제 것을 src/explain_ml.py 로 분리했고, 그 뒤 딥러닝 쪽도 src/explain_dl.py 로 개명해 짝이 맞았습니다. 그림 번호도 01~05 를 비워두고 06-07 을 씁니다. 두 파일은 대상 모델이 달라 둘 다 필요합니다
화면	src/predict.py 는 손대지 않았습니다. ML 모델과 SHAP 근거가 필요하면 explain_ml.explain_one(row) 이 입구입니다. 확률만 필요하면 이유=False 로 11ms, 근거까지면 24ms
전원	imbalanced-learn 이 추가돼 uv.lock 이 바뀌었습니다. 병합 후 uv sync 한 번 돌려주세요

배포 모델 확정 (2026-08-28)

models/ml_model.joblib 을 튜닝 설정 LightGBM 으로 다시 저장했습니다.

ml_meta.json 의 "튜닝": true 로 확인할 수 있고, .gitignore 대상이라

git add -f 로 강제 추가했습니다 (딥러닝 담당이 dl_model.joblib 을 올린 방식과 동일).

부록 — 재현 방법

```
uv sync
```

```
uv run python -m src.train_ml # 기본값 4칸 표 (전체 + 영어)
uv run python -m src.tune_ml # 튜닝 + 불균형 실험 (약 40분)
uv run python -m src.explain_ml # SHAP + 배포 모델 저장
uv run python -m src.demo # 모델링 과정 한 단계씩 보기
```

봉인을 여는 명령은 따로 두었습니다. 실수로 돌아가지 않게 하려는 것입니다.

```
uv run python -m src.tune_ml --봉인측정
```

문서 도구

```
uv run python -m src.verify_docs # 결과서 숫자·경로 검사
uv run --with reportlab python -m src.md2pdf docs/02_학습결과서_머신러닝.md
```

전처리 v1.2 · dataset.csv 139,658행 × 30열 · 이탈률 41.12% · 게임 60개 · SEED 42

탐색 352회 학습 · 총 39분 · 봉인 조각 GroupKFold(5)[0] 고정